

Per-Version Documentation for Ruby Gems

Sources, completeness, and handling variability across versions

1. Problem definition

The unit of analysis is the **(package, version) pair**, not the package. Across 2,814 selected gems there are **202,083 stable versions** — median 30 per gem, mean 72.

Completeness means: for each pair, can documentation be obtained *as it stood at that version*? The answer depends on the source, because sources differ fundamentally in whether they are versioned at all.

2. Version fidelity by source

Source	Versioned	Mechanism	Present in
Doc comments in .gem	Exact	Documentation <i>is</i> the artifact	48.4% of methods
README / docs/ in .gem	Exact	Same archive	78.3% / 5.3%
rubydoc.info	Exact	rubydoc.info/gems/<gem>/<version>	93.4%
GitHub docs/ folder	Conditional	?ref=<tag>, only if tagged	0.6%
GitHub wiki	No	Own history, no release link	81% (mostly empty)
Human-written site	Rarely	Usually current version only	19.8%

Two consequences determine the design.

The in-package tier is versioned by construction. Downloading version 1.2.3's .gem yields version 1.2.3's documentation exactly — not an approximation. There is no completeness gap, only fetching. This covers **96.4% of gems**.

rubydoc.info is redundant. It offers per-version URLs but generates them by running YARD over the same .gem. Scraping costs ~50 requests per gem-version (rack alone has 42 class pages, and no bulk route exists) to recover data already inside one artifact.

3. Tiered acquisition strategy

Rank by version fidelity and record which tier answered, so approximations are never counted as exact.

Tier	Source	Fidelity	Method
1	In-package	Exact	Fetch <gem>-<version>.gem, parse
2	GitHub at tag	Exact if tagged	/contents/docs?ref=<tag>
3	Human site	Current version only	Fetch once, attribute to one version
4	None	—	Record the gap

Tier 1 — apply to everything

```
python count-methods.py --versions all # or minor / major / N
python export-method-inventory.py
rm _docs_scan.csv && python measure-doc-coverage.py
```

measure-doc-coverage.py already keys on <gem>-<version>. Per-version rows appear automatically once the cache holds multiple versions — **no new code required**.

Policy	Fetches	Time @12 workers	Storage
Latest (current state)	2,814	~20 min	1.4 GB
One per major	~8–10k	~1 h	~10 GB
One per major.minor	~30–40k	~4–5 h	~40 GB
Every version	202,083	~26 h	≤250 GB

Tier 2 — tag matching

/repos/{owner}/{repo}/contents/docs?ref=<tag> retrieves a docs folder at any git ref. Gem version 1.2.3 may map to v1.2.3, 1.2.3, rel-1.2.3, <gem>-1.2.3, or no tag. Resolve by ordered candidates, recording which matched:

```
CANDIDATES = ["v{v}", "{v}", "{gem}-v{v}", "{gem}/v{v}",
               "rel-{v}", "release-{v}"]
```

Two failure modes must be recorded rather than smoothed over:

- **Untagged releases** — many gems publish without tagging. No tag, no Tier 2 answer.
- **Monorepos** — rails/rails tags once for seven gems. The tag exists but its docs/ folder is not specific to the queried gem.

Tier 3 — where completeness genuinely breaks

Human-written sites overwhelmingly present one version: the current one. Three sub-cases:

1. **Version switcher present** (/v2.4/..., ReadTheDocs, Docusaurus). Rare but detectable by probing a versioned path.
2. **Site generated from a repo checkout-able at a tag** — already covered by Tier 2.
3. **Site shows only the present** — the large majority.

For case 3, **attribute the site to exactly one version and mark all others as “external documentation, version unknown.”** Attributing today's bundler.io to bundler 1.16 would assert an API that version did not have.

Wayback Machine snapshots can retrieve historical states, but mapping snapshot date to release date is inference, not evidence. If used, record as a distinct tier carrying the snapshot date; never merge with exact tiers.

4. Measuring completeness

Report over (package, version) pairs with the answering tier:

```
coverage_tier ∈ {in_package, github_tag, external_current,
                 external_snapshot, none}
```

Metric	Definition
Exact coverage	Pairs answered by Tier 1 or 2
Approximate	Tier 3 — one version documented, others inferred
Absent	Tier 4

Projected from current data: **~96% exact, ~3% approximate, ~1% absent**. The exact figure is attainable because it depends on no scraping.

Two required properties:

- **Never average across tiers.** “97% documented” is misleading if 3% is inference. Report tiers separately.
- **Record the denominator.** A gem with 500 versions and one measured is 0.2% covered, not “covered”. Store `versions_measured / versions_published` per gem so partial runs remain visible.

5. Analytical value of per-version data

Per-version measurement captures change that latest-only data cannot:

- **Documentation adoption** — a gem that adds docs at v2.0; the event is invisible otherwise.
- **Coverage decay** — new undocumented methods diluting an established gem.
- **Documentation debt over time** — the API census already produces added/removed per version. Joining both series at equal version granularity measures whether documentation tracks or lags API growth.

The third is the novel measurement and requires both series at the same granularity.

6. Recommendation

Use --versions minor — approximately 30–40k fetches, 4–5 hours, ~15% the cost of *all*.

Patch releases are overwhelmingly bug fixes with byte-identical documentation; 26 hours and 250 GB would largely confirm that 1.2.3 and 1.2.4 match. Minor boundaries are where APIs, and therefore documentation, move.

One distortion to control: **sorbet-static has 13,376 versions**, nearly all platform builds of identical code. Such packages would consume a disproportionate share of any *all* run while contributing nothing. `--versions minor` collapses them; `--versions 20` caps them.

Handle the residue explicitly: Tier 2 for the ~100 gems with no in-package documentation but a tagged repository; Tier 3 recorded as single-version for the remainder; Tier 4 counted honestly.

Figures are measured from the 2,814-gem selection in this repository. Timings assume 12 concurrent workers, the rate measured against rubygems.org during collection.